Universidade Federal de Juiz de Fora

Faculdade de Engenharia

Programa de Graduação em Engenharia Elétrica

Pedro Brick Soares

Estudo e implementação de um protocolo de stream de imagem em FPGA

Juiz de Fora

2018

Pedro Brick Soares

Estudo e implementação de um protocolo de stream de imagem em FPGA

Trabalho de Conclusão de Curso apresentada ao Programa de Graduação em Engenharia Elétrica da Universidade Federal de Juiz de Fora, na área de concentração em Eletrônica, como requisito parcial para obtenção do título de Bacharel em Engenharia Elétrica.

Orientador: Luciano Manhães de Andrade Filho

Coorientador: Marcos Vinicius Teixeira

Juiz de Fora

2018

Ficha catalográfica elaborada através do Modelo Latex do CDC da UFJF
com os dados fornecidos pelo(a) autor(a)

Soares,Pedro B..
Estudo e implementação de um protocolo de stream de imagem em FPGA / Pedro Brick Soares. – 2018.
42 f. : il.

Orientador: Luciano Manhães de Andrade Filho
Coorientador: Marcos Vinicius Teixeira
Trabalho de Conclusão de Curso – Universidade Federal de Juiz de Fora, Faculdade de Engenharia. Programa de Graduação em Engenharia Elétrica, 2018.

1. FPGA. 2. CSI-2. 3. Eletrônica. Título.

Pedro Brick Soares

Estudo e implementação de um protocolo de stream de imagem em FPGA

Trabalho de Conclusão de Curso apresentada ao Programa de Graduação em Engenharia Elétrica da Universidade Federal de Juiz de Fora, na área de concentração em Eletrônica, como requisito parcial para obtenção do título de Bacharel em Engenharia Elétrica.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. Luciano Manhães de Andrade Filho - Orientador
Universidade Federal de Juiz de Fora

Me. Marcos Vinicius Teixeira - Coorientador
Universidade Federal de Juiz de Fora

Prof. Dr. Leandro Rodrigues Manso
Universidade Federal de Juiz de Fora

Me. Tiago Aparecido Teixeira
Universidade Federal de Juiz de Fora

AGRADECIMENTOS

Gostaria de agradecer a todos os envolvidos para que fosse possível chegar a este momento.

A todos os colegas que estiveram no meu lado e na minha cabeça. Não importando a situação sempre conseguimos achar uma brecha para rir e tornar os momentos dificeis em aprendizados para a vida.

A todos os professores do curso, desde o ICE até a última aula no campus da engenharia, aos que foram duros e não mudavam de idéia com aquele ponto a mais na prova, além de agradecer, peço desculpas se em algum momento os vi como cruéis e terríveis, hoje percebo, talvez tardiamente, as lições ensinadas e fico grato por tê-los presente na minha formação.

Ao meu professor e orientador Luciano Manhães de Andrade Filho, por todo o conhecimento passado, pelo tempo disponibilizado para me auxiliar neste trabalho e pela indicação ao projeto na empresa *T'allent Interactive*.

Ao coorientador deste trabalho e orientador do projeto que participei na empresa *T'allent Interactive*, Marcos Vinicius Teixeira, agradeço pela oportunidade dada, faltam palavras para agradecer devidamente por todo o conhecimento profissional que agreguei ao participar do projeto e por estar ao seu lado.

Aos meus excelentes pais, dos quais tenho muito orgulho de ser filho, Sérgio Damasceno Soares e Helga Brick Soares, pelo constante suporte e disponibilidade integral para o que fosse preciso. Além de agradecer, peço desculpas pelos problemas gerados, se não fosse pela força dada por vocês, talvez esta hora nunca chegasse.

A minha família, que sempre demonstrou interesse em saber como andava minha graduação e mesmo não estando presente fisicamente, estavam no meu coração.

Obrigado a todos que de certa forma participaram nessa etapa da minha vida.

“A viagem foi longa e o custo foi alto... Mas nunca uma grande conquista foi alcançada facilmente. Um longo conto, como uma torre alta, deve ser construído uma pedra de cada vez”

(Stephen King - A Torre Negra)

RESUMO

Este trabalho apresenta o desenvolvimento de uma propriedade intelectual que será implementada em *FPGA* e descrita em Verilog. Faz parte do objetivo deste trabalho implementar o protocolo CSI-2, visto que este é o mais utilizado em interfaces de câmeras na indústria mobile. Ele tem sido amplamente empregado devido a sua facilidade de uso e habilidade de suportar uma alta taxa de perfomance em fotografias de alta resolução e vídeos em alta definição. De forma a comprovar que o padrão estabelecido pela *MIPI Alliance* é mantido, foi apresentado ao final deste trabalho simulações fazendo uso dos *softwares ModelSim Altera* e *Signal Tap Logic Analyzer*. Os resultados mostraram que a interface implementada seguiu os padrões definidos para uma comunicação serial atuando com alta frequência de operação, justificando assim o seu uso em aplicações que necessitam de uma interface serial para realizar a comunicação entre um sensor e um processador de imagens.

Palavras-chave: FPGA. CSI-2. Eletrônica.

ABSTRACT

This work presents the development of an intellectual property that will be implemented in *FPGA* and described in Verilog. It is part of the objective of this work implement the CSI-2 protocol, since this is the most used camera interface in the mobile industry. It has been widely employed because of its ease of use and the ability to withstand a high performance rate in high resolution photographs and high definition videos. In order to prove that the standard established by the *MIPI Alliance* is maintained it is presented at the end of this work, simulations making use of the *ModelSim Altera* and *Signal Tap Logic Analyzer* softwares. The results showed that the implemented interface followed the standards defined for a serial communication acting with high frequency of operation, thus justifying its use in applications that require a serial interface to perform the communication between a sensor and an image processor.

LISTA DE ILUSTRAÇÕES

Figura 1 – *Interface Paralela padrão de um sensor CMOS* 16
Figura 2 – *Coordenadas Cartesianas do sistema RGB* 17
Figura 3 – *Sistema de cores no modelo YUV* 18
Figura 4 – *Visão geral do Protocolo Low Level* 21
Figura 5 – *Estrutura de um Short Packet* 21
Figura 6 – *Formato do byte DataID* 22
Figura 7 – *Estrutura de um Long Packet* 23
Figura 8 – *Arquitetura de uma FPGA* 25
Figura 9 – *Circuito de um flip-flop D* 26
Figura 10 – *Módulo de um flip-flop D analisado através do RLT viewer* 26
Figura 11 – *Módulo Flip Flop D, destaque para suas portas de entrada e saída* . . . 26
Figura 12 – *TOP LEVEL do circuito CSI-2* 28
Figura 13 – *Circuito que realiza conversão dos dados para padrão digital e rearranja eles no novo formato* 29
Figura 14 – *Padrão dos dados no formato YUV420* 29
Figura 15 – *Circuito BytePacketizer* 30
Figura 16 – *Circuito que agrupa os bytes em pacotes* 31
Figura 17 – *Circuito que anexa os cabeçalhos* 32
Figura 18 – *Demonstração da máquina de estados que comanda o módulo Packet Header* 32
Figura 19 – *Módulo que irá controlar os atrasos necessários para sincronismo* . . . 33
Figura 20 – *circuito representante da conexão com a camada física* 34
Figura 21 – *Módulo serializador de dados* 35
Figura 22 – *Circuitos gerados pelo IP MEGAWIZARD do software Quartus II* . . . 36
Figura 23 – *Imagem retirada do Signal Tap - Dados em paralelo presentes apenas em uma linha de comunicação* 37
Figura 24 – *Imagem retirada do Signal Tap - Dados em paralelo presentes apenas em uma linha de comunicação* 38
Figura 25 – *Imagem retirada do Signal Tap Logic AnalyzerDados entrando em paralelo com os cabeçalhos* 38
Figura 26 – *Imagem retirada do Signal Tap Logic Analyzer - dados do cabeçalho em paralelo e a saída no formato serial* 38
Figura 27 – *Imagem retirada do Signal Tap Logic Analyzer - Dados serializados* . . 39

LISTA DE TABELAS

Tabela 1 – Conversão do formato RGB em YUV 19
Tabela 2 – Tipos de Data Type 22
Tabela 3 – Elementos Lógicos 37

LISTA DE ABREVIATURAS E SIGLAS

FPGA	*Field Programmable Gate Array*
CSI	*Camera Serial Interface*
HDL	*Hardware Description Language*
CFA	*Color Filter Array*
LVDS	*Low Voltage Differential Signaling*
ADC	*Analogic Digital Converter*
CMOS	*Complementary Metal-Oxide-Semiconductor*
CCD	*Charge-Coupled Device*
PAL	*Phase Alternating Line*
NTSC	*National Television System(s) Committee*
CCI	*Camera Control Interface*
SoT	*Start of Transmission*
EoT	*End of Transmission*
LLP	*Low Level Protocol*

SUMÁRIO

1 INTRODUÇÃO 12
1.1 Motivação 12
1.2 Objetivo 13
1.3 Visita Guiada 14

2 Revisão Bibliográfica 15
2.1 Sensor de imagem RGB 15
2.2 Espaço de Cores 17
2.2.1 Sistema de Cores RGB 17
2.2.2 Sistema de Cores YUV 17
2.2.3 Conversão RGB - YUV 18
2.3 **Protocolo CSI-2** 19
2.3.1 Especificação D-PHY 19
2.3.2 Especificação CSI-2 19
2.3.2.1 **Camadas Presentes** 20
2.4 *Low Level Protocol* 20
2.4.1 Campos presentes no Low Level Protocol 21
2.4.1.1 Formato de um Short Packet 21
2.4.1.2 Formato de um Long Packet 22
2.4.1.3 *Word Count* 23

3 Implementação 24
3.1 Arquitetura 24
3.1.1 Linguagem Descritiva de Hardware 25
3.1.1.1 Verilog 25
3.2 Descrição da interface desenvolvida 26
3.2.1 Visão Geral do Circuito 26
3.2.2 Circuito conversor do formato RGB para o padrão digital YUV 29
3.2.3 Módulo responsável por agrupar os bytes de dados em pacotes 30
3.2.3.1 Circuito para aglomerar os bytes 30
3.2.3.2 Circuito para adicionar cabeçalhos 31
3.2.4 Circuito controlador de atrasos 33
3.2.5 Circuito representante da camada física 33
3.2.5.1 Circuito serializador dos dados 34
3.2.5.2 Circuito para realizar conversão do formato serial para diferencial 35

4 Resultados 37
4.1 Simulação com o Signal Tap 37
4.1.1 Interface com 1 linha de comunicação 37
4.2 Interface com 2 linhas de comunicação 38

5 Conclusão 40
5.1 Trabalhos Futuros 40

REFERÊNCIAS 41

REFERÊNCIAS 42

1 INTRODUÇÃO

Nos dias atuais, cada vez mais pode-se notar a influência dos avanços tecnológicos. O aumento do uso de sensores em diversas aplicações, torna necessário o surgimento de uma interface de comunicação que seja capaz de transmitir os dados dos sensores para os microcontroladores e os dispositivos lógicos programáveis.

Para a industria eletrônica, é vital que novos produtos cheguem ao mercado consumidor no menor espaço de tempo possível. Desta forma, a redução do tempo de desenvolvimento de novos projetos tornou-se um fator de grande importância para os fabricantes de dispositivos eletrônicos. Além de prejuízos de mercado, um longo tempo de desenvolvimento implica também na elevação dos custos de projeto.

O surgimento de dispositivos reconfiguráveis como os FPGAs, abriram novas possibilidades. Hoje, é possível desensolver rapidamente um projeto de sistema digital empregando-se novas metodologias como linguagens de descrição de hardware (HDLs, do inglês *Hardware Description Language*), ferramentas de síntese lógica e simulação.

Como os sistemas reconfiguráveis geralmente são constituídos por matrizes de portas programáveis (FPGAs, do inglês Field Programmable Gate Array), a utilização destes na indústria tem um amplo horizonte, podendo implementar desde uma unidade lógica elementar até mesmo desempenhar o papel de interface com outros hardwares, que eventualmente irão compor um sistema complexo. No caso dos sistemas que exigem a transferência de imagens por protocolos específicos, essa característica é de suma importância, pois permite implementar a comunicação entre componentes de captura e processamento.

O processamento de imagens capturadas pode ser realizado tanto na própria FPGA quanto em um processador externo. Neste último caso, a FPGA tem como função atuar como um gerenciador, compatibilizando a saída de dados do componente de captura à entrada de dados do componente de processamento. Com esta solução obtém-se flexibilidade quanto à utilização de outros componentes no sistema como por exemplo, a entrada de dados, além de possibilitar customizações apenas no circuito incorporado em FPGA, permitindo que o hardware de processamento não sofra modificações (Petroni,2015).

1.1 Motivação

Sensores de imagem CMOS convencionais foram tradicionalmente equipados com uma interface paralela que utiliza sinalização unidirecional para codificar todos os dados de vídeo necessários. Esta interface paralela, contém um certo número de linhas de dados paralelos com um clock correspondente e sinais de sincronização. As configurações típicas são de 8 ou 12 linhas de dados, equivalentes a resolução bit pixel do sensor, onde cada

linha individual representa um único bit de o pixel da imagem e os pixels completos são transmitidos em cada borda de clock do pixel correspondente. A vantagem é óbvia: permite uma interface muito simples com baixo receptores de complexidade.

O aumento da resolução dos sensores, largura de banda e os requerimentos de integridade de sinal levaram ao uso de interfaces seriais, como por exemplo o LVDS (do inglês, *low voltage differential Signaling*). Embora transmissores seriais necessitem de uma frequência de clock maior, eles permitem uma melhoria significativa na integridade do sinal de todo o sistema. A interface diferencial serial implementada em sensores de imagem substitui os sinais unidirecionais da interface paralela com um clock diferencial por linha e uma ou mais linhas de dados serializados e no modo diferencial. Ao usar dados diferenciais e serializados, o número de pinos I/O é reduzido, diminuindo a complexidade de placas a serem projetadas.

Desta forma, junto com o aumento da demanda por sensores com melhor resolução de imagem e taxas de frame mais rapidas surgiram novas interfaces, como as especificações CSI (do inglês, *camera serial interface*) substituindo as paralelas, estabelecidas pela MIPI Alliance.

O protocolo para CSI-2 é o mais usado em interfaces de câmeras na indústria *mobile*. Ele tem sido amplamente utilizado devido a sua facilidade de uso e habilidade de suportar uma alta taxa de perfomance em fotografias de alta resolução e vídeos em alta definição. Ele é caracterizado por um padrão de transmissão de dados unidirecional serial e uma interface de controle entre transmissor e receptor. As suas especificações são separadas em duas: o meio físico de transmissão e as interfaces e protocolos dos pacotes a serem transmitidos, criando uma liberdade que permite que qualquer pacote seja transportado entre suas camadas até o receptor desde que este esteja nos padrões conhecidos e especificados pelo próprio protocolo.

A MIPI ALLIANCE promove suas especificações CSI como padrões abertos, embora se deva observar que todas as especificações do CSI são confidenciais, portanto o grau real de abertura e risco de violação destas não são claras.

A partir disto, é justificado o desenvolvimento de aplicações específicas para novos projetos. Assim, cada vez mais vão surgindo novas interfaces de comunicação entre sensores e chips.

1.2 Objetivo

Este trabalho apresenta o desenvolvimento de uma propriedade intelectual que será implementada em uma *FPGA* e descrita em verilog, com a finalidade de servir como interface de câmeras em sistemas mobile. Neste trabalho, o processamento de imagem será realizado por um *hardware* externo e a comunicação entre ele e a FPGA será demonstrada

também.

Os resultados serão apresentados na forma de gráficos de simulação e comparados com os padrões estabelecidos pela *MIPI ALLIANCE*, afim de obter uma interface com um comportamento similar porém específica para a aplicação desejada.

1.3 Visita Guiada

O trabalho será dividido em 5 capítulos, no capítulo 2 é apresentada uma revisão bibliográfica, fazendo referência a trabalhos nessa area de atuação, e a descrição protocolo utilizado.

No capítulo 3 será abordada a implementação em *FPGA*, descrevendo a arquitetura da mesma, a linguagem descritiva de hardware utilizada e o processador desenvolvido.

No capítulo 4 são apresentados e comentados os resultados, que serão simulações a partir do *SignalTap*.

Por fim, o capítulo 5 apresenta as conclusões do trabalho e possíveis aperfeiçoamentos para outras aplicações do mesmo nível.

2 Revisão Bibliográfica

O protocolo apresentado neste trabalho é o CSI-2, ele caracteriza um padrão de transmissão de dados bidirecional serial e uma interface de controle entre transmissor e receptor. As suas especificações são separadas em duas: o meio físico de transmissão e as interfaces e protocolos dos pacotes a serem transmitidos, criando uma liberdade que permite que qualquer pacote seja transportado entre suas camadas até o receptor desde que este esteja nos padrões conhecidos e especificados pelo próprio protocolo.

A propriedade intelectual mencionada, fornece ao sensor de imagem uma interface *low power,high speed*, de baixo custo que suporta uma quantidade extensa de soluções para imagens em aparelhos móveis, pc-câmera e dispositivos de gravação de vídeos em veículos.

Devido ao alto preço de um *core* CSI-2, foi necessário desenvolver uma propriedade intelectual própria com as mesmas características, a fim de reduzir o custo do projeto.

2.1 Sensor de imagem RGB

Nos dias atuais, cada vez mais pode-se notar a influência dos avanços tecnológicos. É perceptível o aumento de sistemas embarcados e integrados a sistemas digitais de imagem. Estes fazem uso de sensores de imagem, e neste trabalho foi abordado a interface que comunica o sensor com um processador.

O conceito básico de um sensor de imagem é converter a luz incidente ou a imagem em uma representação digital. O processo em si começa com uma ou mais lentes ópticas sendo focadas e ajustadas para a cena desejada. A luz passa por um filtro matricial de cores (CFA, do inglês Color Filter Array) que contém um padrão pré-definido de cores e então é convertida em um sinal de domínio analógico por photodetectores posicionados atrás do filtro.

O filtro matricial de cores é usado para realizar imagens coloridas. Ao usar este tipo de filtro, cada pixel irá produzir um sinal correspondente a uma das cores representadas na matriz. Os pixels em formato de dados analógicos são lidos e digitalizados com a ajuda de um conversor analógico para digital (ADC, do inglês Analogic Digital Converter) e processados por um chip externo ao sistema de imagem. A fim de produzir uma imagem colorida por completo, uma interpolação espacial conhecida como demosaicing é usada (Etheredge, 2016 e Gamal, 2005).

Atualmente, existem dois tipos mais comuns de sensores de imagem, CMOS (do inglês, complementary metal-oxide semiconductors) e CCD (do inglês, charge-coupled devices). Embora os sensores CCD são associados a uma qualidade melhor, estes também tem um preço mais alto no mercado, de forma que os sensores CMOS vem crescendo em popularidade, além de ter um baixo custo alguns modelos já conseguem equiparar em

termos de qualidade com os CCD (Mehta, 2015).

O sensor de imagem, representa a primeira parcela do sistema final no qual consiste esse trabalho. Ele será conectado a uma FPGA, que interpretará os dados em RGB e prepará-los para o processador final. É importante ressaltar então, como é feita essa união física entre o sensor e a FPGA.

Os sensores de imagem do tipo CMOS possuem uma interface paralela que utiliza sinais unidirecionais para codificar os dados de vídeo necessários. Esta interface que pode ser vista na figura abaixo contém um certo número de linhas de dados paralelos, com sinais de clock e sincronização específicos. A vantagem da interface paralela é bem clara: permite uma comunicação simples com recebedores de baixa complexidade.

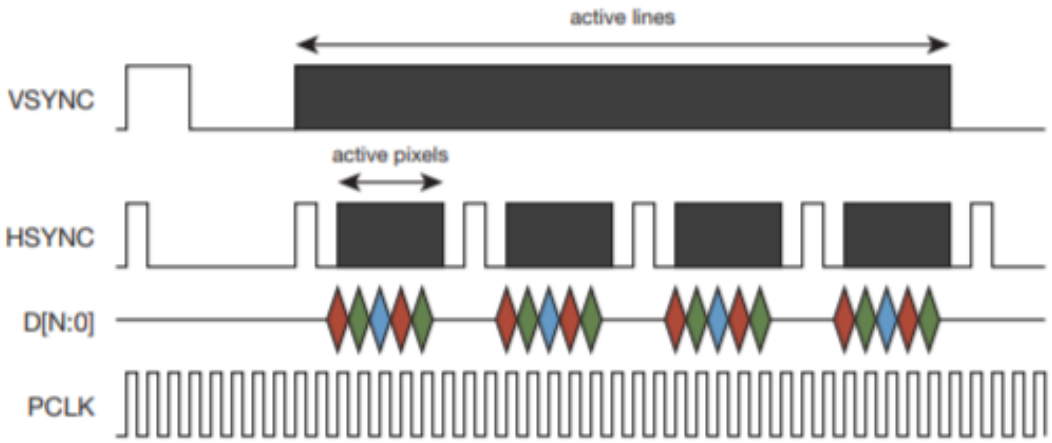

Figura 1 – *Interface Paralela padrão de um sensor CMOS*

O aumento da resolução dos sensores, largura de banda e os requerimentos de integridade de sinal levaram ao uso de interfaces seriais, como por exemplo o LVDS (do inglês, *low voltage differential Signaling*). Embora transmissores seriais necessitem de uma frequência de clock maior, eles permitem uma melhoria significativa na integridade do sinal de todo o sistema. A interface diferencial serial implementada em sensores de imagem, substitui os sinais unidirecionais da interface paralela com um clock diferencial por linha, e uma ou mais linhas de dados modo diferencial e serializados. Ao usar dados neste formato, o número de pinos I/O é reduzido, diminuindo a complexidade de placas a serem projetadas.

Desta forma, junto com o aumento da demanda por sensores com melhor resolução de imagem e taxas de frame mais rápidas, surgiram novas interfaces, como as especificações CSI (do inglês, *camera serial interface*) substituindo as paralelas, estabelecidas pela MIPI Alliance.

2.2 Espaço de Cores

Um espaço de cores é uma representação matemática de um conjunto de cores. Existem diversos espaços de cores, dentre eles podemos citar o modelo RGB(red, green e blue ou vermelho, verde e azul) que é utilizado em computadores e o modelo YIQ,YUV ou YcbCr usado em sistemas de vídeos.

2.2.1 Sistema de Cores RGB

O sistema de cores RGB é um modelo aditivo, descrevendo as cores como uma combinação das três primárias: vermelho (**R**ed), verde (**G**reen) e azul (**B**lue). Qualquer cor no sistema digital é representada por um conjunto de valores numéricos. O modelo RGB por exemplo, pode se representado por valores inteiros de 0 a 255.

Como é um modelo aditivo, a cor branca corresponde à representação simultânea das três primárias (1,1,1) enquanto a cor preta corresponde à ausência das mesmas (0,0,0). A figura especifica as coordenadas cartesianas do sistema RGB, onde a sua diagonal equivale aos tons de cinza, originados pela mesma intensidade nas três cores.

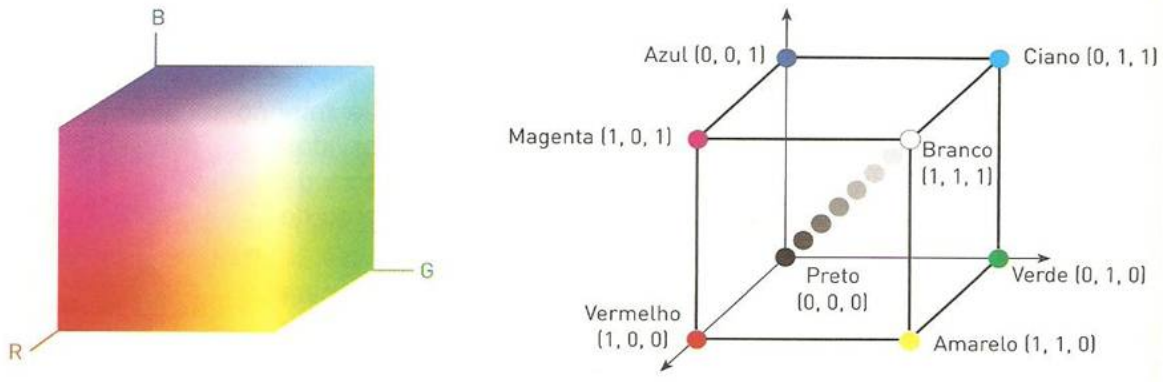

Figura 2 – *Coordenadas Cartesianas do sistema RGB*

O sistema RGB não é o mais eficiente para fazer o processamento de imagens. Por exemplo, para modificar a intensidade ou a cor de um determinado pixel é necessário ler as três cores do buffer da imagem, fazer as modificações necessárias e gravar novamente as três cores no buffer. Por essa e outras razões, muitos sistemas de vídeo usam luminosidade e duas diferenças entre cores. Os sistemas mais comuns que utilizam esse padrão são YUV, YIQ, YCbCr.

2.2.2 Sistema de Cores YUV

O sistema de cores YUV é usado pelos padrões de sistema de televisão em cores PAL (do inglês, *Phase Alternation Line*), NTSC (do inglês, *National Television System*

Committee) e SECAM (do francês, *Sequetiel Couler Avec Mémorie*). O espaço YcbCr foi desenvolvido como parte da resolução ITU-RBT.601, que visa criar um padrão mundial para uma componente de vídeo digital. O padrão é uma codificação digital para o espaço YUV e as três componentes têm por definição 8 bits.

Este padrão concentra a maior parte da informação da imagem na componente referente a luminância (Y) e menos informação na Chrominância (U e V), permitindo que a chrominância seja menos especificada do que a luminância.

Na figura 3, pode ser visto como as cores podem ser codificadas através das componentes U e V ,e a componente Y representando a intensidade do sinal.

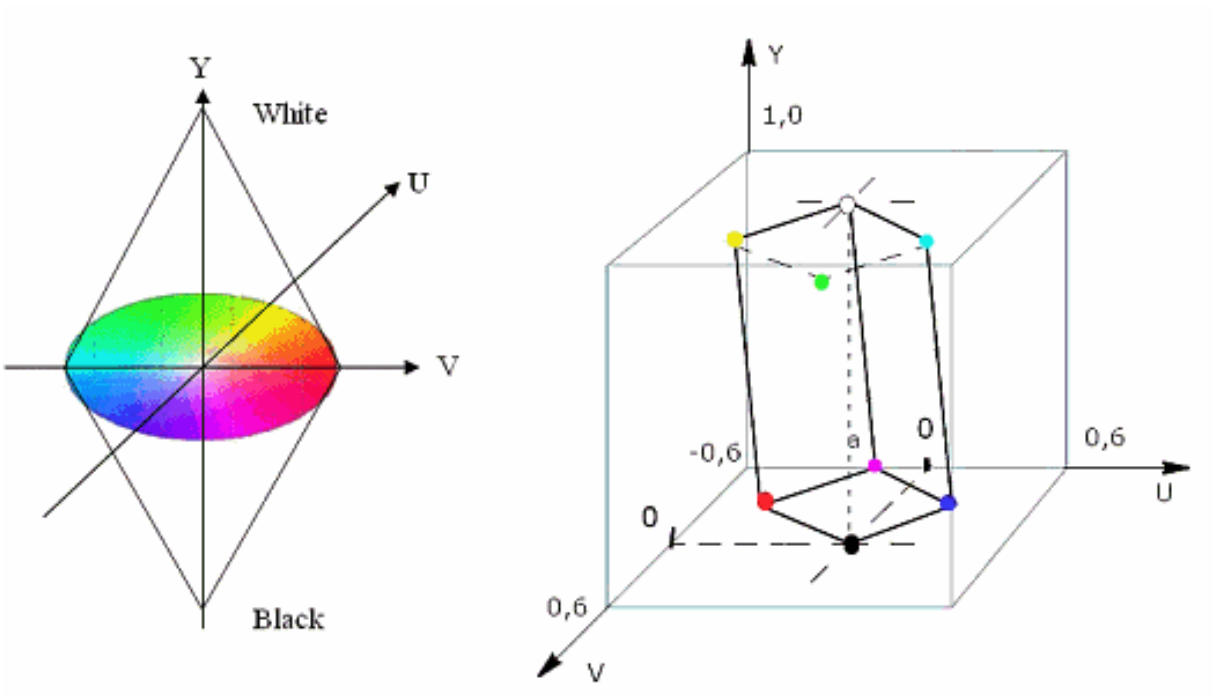

Figura 3 – *Sistema de cores no modelo YUV*

Existem diversos formatos de YcbCr tais como 4:4:4, 4:2:2, 4:1:1 e 4:2:0. Nesses formatos, a primeira componente é referente a luminância, a segunda representa a crominância Azul e a terceira a crominância vermelha.

2.2.3 Conversão RGB - YUV

A redução de espaço utilizado ou de fluxo de dados usando o YUV no lugar do RGB é dada por um fator de 1.5. Se o formato RGB é definido por 8 bits para cada cor, cada pixel do RGB é represetando por 24bits, no caso do YUV, cada pixel é descrito em média por 16bits, 8 bits para a luminância e 8 bits para chrominancia por pixel.

Como a maioria dos displays usa o formato RGB e a maioria dos processadores de imagem usam o formato mais compacto YUV, uma conversão de cor é necessária. A

conversão atual entre o padrão RGB e o formato YUV é diferente para sinais digitais e sinais analógicos.

Para o caso deste trabalho, foi utilizada a tabela abaixo para realizar a conversão dos formatos.

Tabela 1 – Conversão do formato RGB em YUV

$Y =$	0,299R + 0,587G + 0,114B
U =	-0,147R -0,289G + 0,436B
V =	0,615R -0,515G -0,100B

2.3 Protocolo CSI-2

O protocolo CSI-2 fornece ao sensor de imagem uma interface *low power, high speed*, de baixo custo que suporta uma quantidade extensa de soluções para imagens em aparelhos móveis, câmera de computadores e dispositivos veiculares de gravação de videos.

O CSI-2 caracteriza um padrão de transmissão de dados e interface de controle entre transmissor e receptor. A interface de transmissão de dados é unidirecional e serial.

A interface de controle (CCI, do inglês Camera Control Interface) é bi-direcional e compatível com os padrões I2C. As definições da *MIPI Alliance* são feitas em duas especificações separadas:

- Especificação D-PHY, referente ao meio de transmissão
- Especificação CSI-2, referente a interface de controle e protocolo de transmissão

2.3.1 Especificação D-PHY

A camada física especifíca o meio de transmissão (condutores elétricos) e os circuitos de entrada e saída. Os dados podem ser unidirecionais ou bi-direcionais e a conexão inclui um modo de sinal *HIGH-SPEED (HS)* para um tráfego de dados rápidos, e um sinal *LOW-POWER (LP)* para controle. Essa parte da especificação documenta as características de transmissão, os parâmetros elétricos para os sinais e a relação de temporização entre os clocks e os dados nas lanes.

2.3.2 Especificação CSI-2

A especificação CSI-2 define as interfaces de controle e o protocolo de transmissão de pacotes, no qual informa a arbitrariedade dos dados a serem transportados do transmissor para o receptor. É um protocolo orientado por byte, que transporta dados usando um pacote *Short* e um pacote *Long*.

Estes dois são usados para comunicação com um protocolo *LOW-LEVEL* e cada pacote é diferenciado pelo campo *data type* presente no seu cabeçalho. Composta por várias outras camadas, cada uma com responsabilidades próprias, o protocolo CSI-2 possibilita múltiplas *streams* de dados usando uma única *interface* no *host*. Esta camada determina como as múltiplas *streams* devem ser marcadas e intercaladas para que cada uma possa ser reconstruída de forma apropriada.

2.3.2.1 Camadas Presentes

1. Camada de Pixel/Byte Packing/Unpacking

   O CSI-2 suporta vários tipos de formatos de pixels em suas aplicações, de 6 até 24bits por pixel. No transmissor, esta camada empacota os pixels da aplicação em bytes antes de encaminhar o dado para a camada *Low Level*. No receptor, esta desempacota os bytes provenientes do *Low Level Protocol* em pixels, antes de mandá-los para a camada de aplicação. Sempre serão 8bits por pixel a serem transferidos nesta camada.

2. Gerenciamento de Lanes

   O CSI-2 pode ter múltiplas lanes, variando de 1 até 4 depedendo da demanda da largura de banda da aplicação. O lado transmissor da interface distribui os dados de saída pelas lanes, e no lado receptor, a interface coleta os bytes provenientes das lanes e reorganiza eles de forma a recombinar a stream de dados com a sequência inicial.

3. Low Level Protocol

   Estabelece meios para sincronização a níveis de bit e bytes para transferência serial de dados entre os eventos de ínicio de transmissão (do inglês,*Start of Transmission*) e fim de transmissão (do inglês, *End of Transmission*). Será descrito com mais detalhes na próxima secção.

4. Camada de Aplicação

   Descreve a codificação e interpretação de nível mais alto de dados contidos na stream. A especificação CSI-2 determina o mapeamento de valores de pixels em bytes.

2.4 *Low Level Protocol*

O protocolo Low Level(LLP) é orientado por byte e baseado em pacotes. Ele efetua o transporte de dados arbitrários usando os pacotes no formato *short* e *long*. Suas principais características são:

- Transporte de dados arbitrários: Os dados estando corretamente codificados no formato do protocolo são transportados para as demais camadas.
- Palavras de 8 bits : todos os dados presentes na camada referente ao protocolo tem o mesmo número de bits por byte, criando um padrão para auxiliar na decodificação.
- Pacotes especiais para informar o início e o fim de um frame, assim como o início e o término de uma linha de dados.
- 16 bits Checksum : Para detecção de erros, este parte do protocolo é opcional.

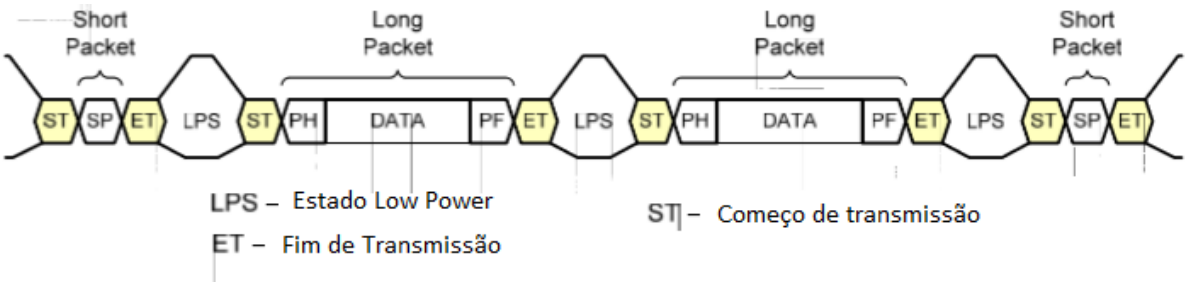

Figura 4 – *Visão geral do Protocolo Low Level*

2.4.1 Campos presentes no Low Level Protocol

Para a comunição, duas estruturas são definidas no Low Level Protocol, *Short Packet* e *Long Packet*. O início de cada é marcado pela transição da saída do estado de *Low Power* para a sequência de começo de transmissão (SoT), e o fim de cada pacote é marcado pela transição da sequência de fim de transmissão (EoT) para o estado de *Low Power*.

2.4.1.1 Formato de um Short Packet

Um Short Packet será identificado pelos valores de Data Type entre 0x00 até 0x0F. Ele deverá conter apenas um *Packet Header*.

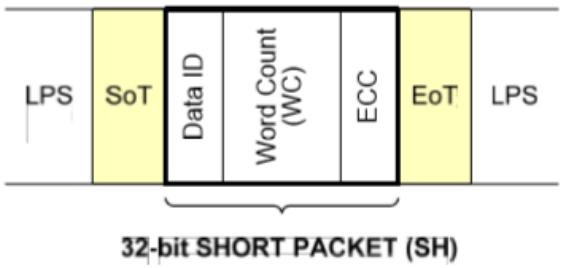

Figura 5 – *Estrutura de um Short Packet*

Para um *Data Type* de sincronização de frames, o campo de dados do *Short Packet* deverá ser o número de frames, para um de sincronização de linhas, o campo de dados do *Short Packet* deverá ser o número de linhas e para *Data Types* de *Short Packet* genérico, o conteúdo do pacote deve ser definido pelo usuário.

O byte de ECC permite erros em um bit serem corrigidos ou erros de 2-bits serem detectados no Short Packet.

O byte de *Data Identifier* contém o identificador de canal virtual (VC) e o valor do *Data Type*. O VC é localizado nos dois bits mais significativos enquanto o DT é contido nos seis bits menos significativos do byte.

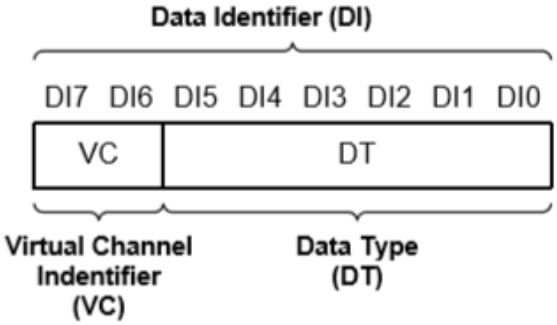

Figura 6 – *Formato do byte DataID*

Os bits de Data Type podem assumir os seguintes valores:

Tabela 2 – Tipos de Data Type

Data Type	Descrição
$0x00 - 0x07$	Data Type para sincronização de pacotes Short
0x08 - 0x0F	Data Type genérioco para pacotes Short
0x10 - 0x17	Data Type genérico para pacotes Long
0x18 - 0x1F	DADOS YUV
0x20 - 0x27	DADOS RGB
0x28 - 0x2F	DADOS RAW
0x30 - 0x37	BYTES DE DADOS DEFINIDOS PELO USUÁRIO
0x38 - 0x3F	Reservado

2.4.1.2 Formato de um Long Packet

Um *long packet* será identificado através do *Data Type*, este variando de 0x10 até 0x37. Um pacote deste tipo consiste de 3 elementos: 32-bit *Packet Header* (**PH**), dados da aplicação e 16-bit *Packet Footer* (**PF**). O *Packet Header* é composto por 3 elementos: 8-bit Data ID, 16-bit Word Count e 8-bit ECC. O *Packet Footer* é composto por um único elemento: 16-bit checksum.

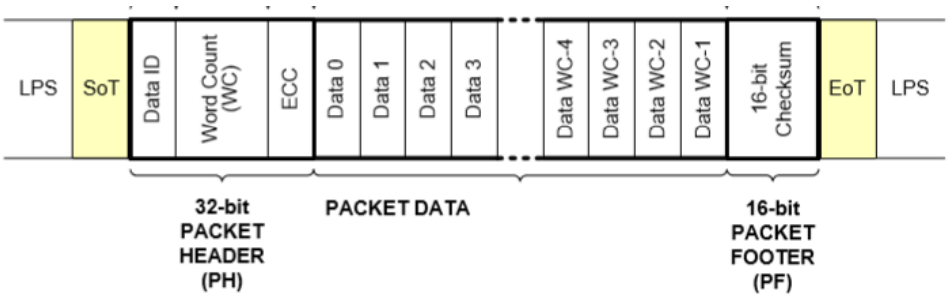

Figura 7 – *Estrutura de um Long Packet*

2.4.1.3 *Word Count*

O *Word Count* determina o número de palavras de 8-bit na carga útil dos dados, entre o fim do *Packet Header* e o começo do *Packet Footer* por fim, o byte de ECC permite erros em um bit serem corrigidos ou erros de 2-bits serem detectados no Packet Header.

Após o fim do Packet Header, o receptor lê as próximas palavras de 8-bits contendo dados e enquanto isto ocorre, o receptor não deve procurar por códigos de sincronização embutidos, desta forma não há limitações para o valor de uma palavra de dados, visto que esta pode estar separadas em blocos de 8-bits. Assim que o receptor termina de ler os dados, ele prossegue para leitura do checksum no *Packet Footer*, cada bit deve ser transmitido com o primeiro sendo o menos significativo. A *stream* inteira pode ser transmitida em qualquer ordem de bytes restringida apenas pelos requerimentos do formato. Depois que a sequência de fim de transmissão (EoT) acaba, o receptor começa a procurar pela nova sequência de início de transmissão (SoT).

3 Implementação

A tecnologia envolvida nos circuitos digitais vem se desenvolvendo cada vez mais rápido nas últimas décadas. Devido a isso, o modo de projetar hardware vem se transformando aceleradamente. Com o advento da microeletrônica, componentes de circuitos digitais como transistores foram aperfeiçoados para circuitos integrados. A criação de chips de memória e microprocessadores contribuíram para consolidação de um novo mercado, o de dispositivos lógicos programáveis.

Um dos tipos de dispositivos lógicos programáveis com capacidade de implementar circuitos digitais é o **FPGA**. Ele é utilizado em vários setores industriais onde desempenho, paralelismo e tempo real são fundamentais.

Introduzido pela empresa Xilinx Inc. no ano de 1985, o *Field-Programmable Gate Array* (**FPGA**) é um dispositivo lógico programável que suporta a implementação de circuitos lógicos relativamente grandes. Os FPGAs não possuem planos de portas *OR* ou *AND*, em vez disso, estes componentes consistem de um grande arranjo de células configuráveis (ou blocos lógicos) que podem ser utilizadas para a implementação de funções lógicas.

Um FPGA possui três tipos principais de recursos: blocos lógicos, blocos de entrada e saída (I/O), e chaves de interconexão. Os blocos lógicos formam um arranjo bidimensional, e as chaves de interconexão são organizadas como canais de roteamento horizontal e vertical entre as linhas e colunas de blocos lógicos. Estes canais de roteamento possuem chaves programáveis que permitem conectar os blocos lógicos de maneira conveniente, em função das necessidades de cada projeto.

Estes dispositivos podem ser usados para agregar sensores de imagem com sistemas de vídeos embarcados, sem necessidade de alterar o hardware do sistema. Para realizar a interface entre o sensor e o sistema existem as especificações da MIPI ALLIANCE para **CSI** (*camera serial interface*), atualmente contando com as versões 1,2 e 3 o CSI promove um link de alta velocidade com ótima eficiência energética e compatível com a maioria dos processadores (Noronha, 2005).

3.1 Arquitetura

Ao possuir uma arquitetura reconfigurável, estes dispositivos executam processamento em paralelo, envolvendo diversas unidades funcionais afim de, diminuir o tempo de resposta e aumentar o desempenho de execução dos conjuntos de instruções, permitindo a customização da capacidade computacional da máquina de acordo com a aplicação. A arquitetura de roteamento de um FPGA é a forma pela qual os seus barramentos e as chaves de comutação são posicionadas para permitir a interconexão entre as células

lógicas. Este tipo de arquitetura permite que alguns FPGAs possam ser configurados e reconfigurados várias vezes.

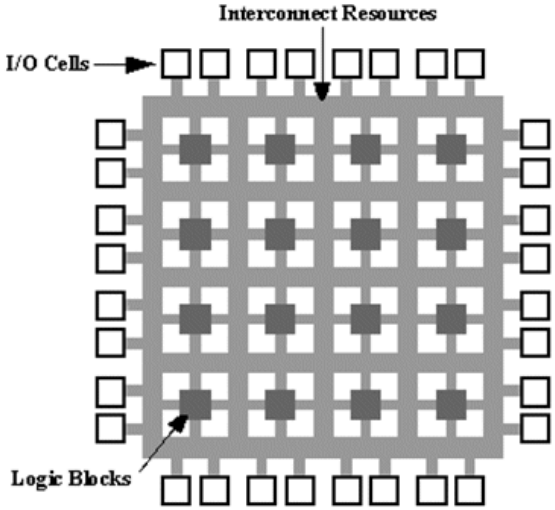

Figura 8 – *Arquitetura de uma FPGA*

3.1.1 Linguagem Descritiva de Hardware

A linguagem de descrição de hardware é utilizada para descrever o comportamento de um sistema digital de várias formas como, equações lógicas, tabelas verdades e máquinas de estados.

Entre os tipos de linguagem descritiva de hardware existentes nos dias de hoje, para este trabalho foi escolhido usar o Verilog.

3.1.1.1 Verilog

O verilog permite representar a hierarquia de um projeto, através de duas estruturas:

- Módulos

  São projetados para executar as funções necessárias do sistema, em um sistema hierárquico, cada módulo terá uma posição, podendo ter mais de um com o mesmo nível de hierarquia. Na figura 8, pode ser visto como é construído um circuito de um flip-flop D em verilog.

  O mesmo módulo, depois de compilado, pode ser analisado da seguinte forma.

- Portas

  São estruturas onde os dados passam de um módulo para o outro. Estão presentes em todos os módulos, nas formas de entradas e saídas. Na figura 10, em destaque o módulo que está em um nível hierárquico maior que o anterior.

```
module ff_d
(
input clk,rst,entrada,
output reg saida
);

always @ (posedge clk) begin
if(rst == 1'b0)begin
saida <= 1'b0;
end
else begin
saida  <= entrada;
end
end

endmodule
```

Figura 9 – *Circuito de um flip-flop D*

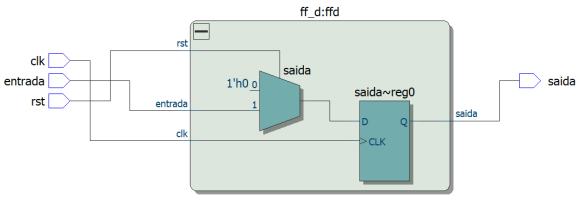

Figura 10 – *Módulo de um flip-flop D analisado através do RLT viewer*

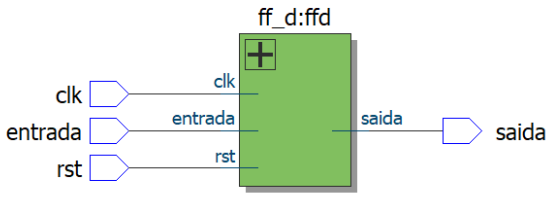

Figura 11 – *Módulo Flip Flop D, destaque para suas portas de entrada e saída*

3.2 Descrição da interface desenvolvida

Nesta seção são apresentados os circuitos desenvolvidos que ao receber os dados de um sensor de imagem rgb, servirão de auxílio para um processador de imagem trabalhar com as informações captadas.

3.2.1 Visão Geral do Circuito

O codificador foi feito de forma que tenha três entradas principais, sendo elas as componentes RGB fornecidas por um sensor de imagem. O circuito funcionará convertendo esses valores em seus equivalentes no formato YUV420, e após essa conversão encaminhará

os dados para serem serializados e ordenados no forma desejada. Ele é composto por :

- Ciruito conversor do padrão de imagem RGB para o formato digital YUV
- Circuito que irá agrupar os bytes de dados em pacotes de 32bits
- Circuito para adicionar um delay necessário para que os dados estejam sincronizados
- Circuito representante da camada física, onde os dados serão serializados e convertidos para o formato diferencial de transmissão.

A saída final, serão os bits individuais no formato diferencial, separados em um número fixo e predeterminado de linhas. A transmissão ocorrerá tanto na borda de subida quanto na borda de descida do pulso de clock.

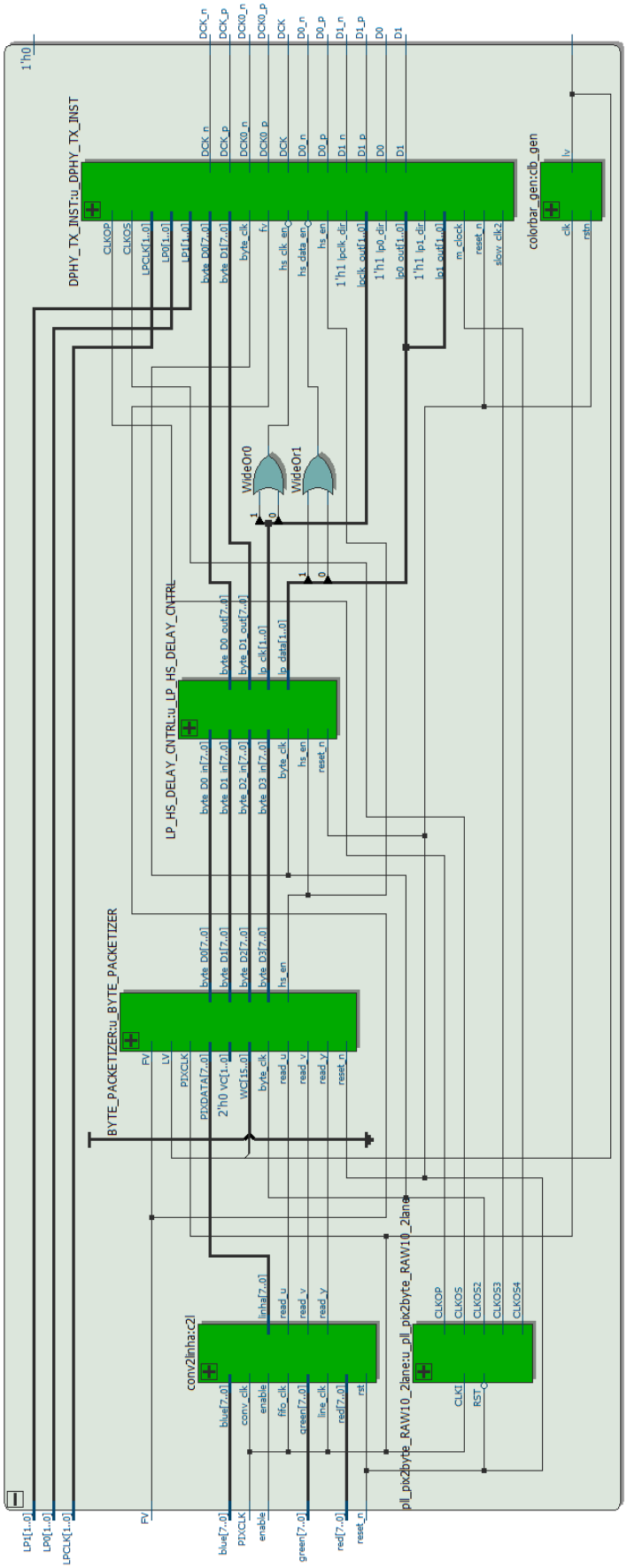

Figura 12 – *TOP LEVEL do circuito CSI-2*

3.2.2 Circuito conversor do formato RGB para o padrão digital YUV

Este é o primeiro circuito do codificador. Sua função é realizar a conversão dos dados fornecidos pelo sensor de imagem RGB em YUV420. Após esta conversão, os dados são armazenados em uma memória do tipo FIFO e em seguida encaminhados para o próximo circuito.

Na figura 13, os subcircuitos que compõem o circuito que irá converter as componentes RGB para o padrão YUV e ordená-las no novo formato.

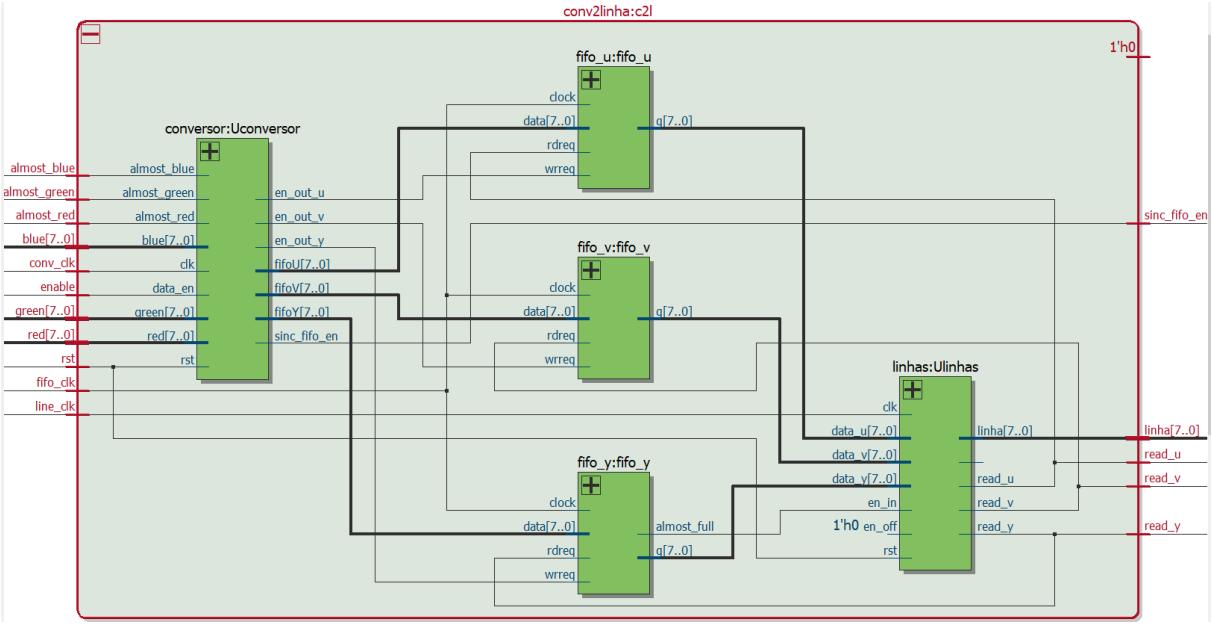

Figura 13 – *Circuito que realiza conversão dos dados para padrão digital e rearranja eles no novo formato*

No módulo de nome Linhas, os dados são organizados seguindo o padrão de imagem YUV420.

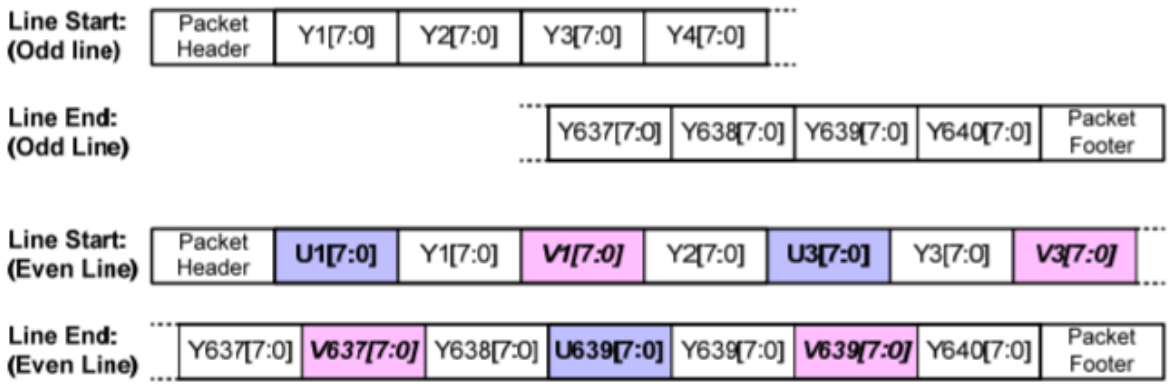

Figura 14 – *Padrão dos dados no formato YUV420*

3.2.3 Módulo responsável por agrupar os bytes de dados em pacotes

Este circuito é composto por 2 circuitos internos, sendo eles:

- Um circuito que irá agrupar os bytes em pacotes
- Um circuito responsável por adicionar o cabeçalho específico de cada pacote
- Ainda existe a opção de um terceiro circuito, Crc16, porém é um circuito opcional no CSI-2 e para evitar o uso excessivo de recursos da FPGA, ele não está sendo utilizado.

Este módulo tem como função converter dados que entraram de forma paralela em pacotes de bytes, e atribuir ao conjunto de bytes um cabeçalho contendo as informações referentes ao formato da imagem, feito isso, o circuito irá dispor os dados em até 4 lanes. No caso escolhido, 2 lanes, o circuito terá duas saídas sem dados e duas saídas com os dados da aplicação. Estas informações são necessárias para que o receptor compreenda os dados que está recebendo e possa decodifica-los de acordo com o padrão correto.

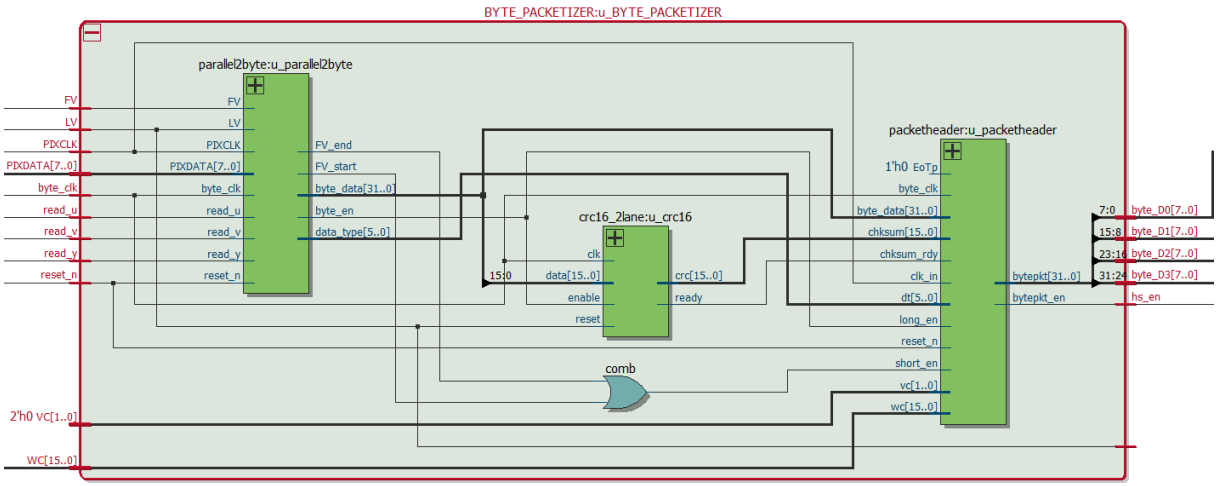

Figura 15 – *Circuito BytePacketizer*

3.2.3.1 Circuito para aglomerar os bytes

Este circuito tem como objetivo agrupar os dados em pacotes de até 32bits, variando de acordo com o número de lanes escolhidas para trabalhar. Neste trabalho, foi escolhido trabalhar com 2 lanes, então os pacotes deste circuito serão de 16bits. Estes serão formatados do bit mais significativo para o menos significativo.

O bloco contém também a função de indicar quando ocorre o começo e o fim de um frame, e através de um parâmetro pré-determinado, o circuito informa qual o padrão

de imagem que está sendo seguido, através da variável *data_type*. O controle é feito pelo circuito denominado MONO, que consiste de duas portas lógicas do tipo *AND*. Ele definirá o estado da variável FV baseado no dado de entrada da mesma e sua saída marcará o começo ou o fim de um frame da imagem.

Na figura 16, em destaque os circuitos que compõe o subcircuito Parallel2byte, com destaque para o circuito MONO, o circuito que irá separar os dados em pacotes de até 32bits e a memória que armazenará a stream proveniente do circuito anterior.

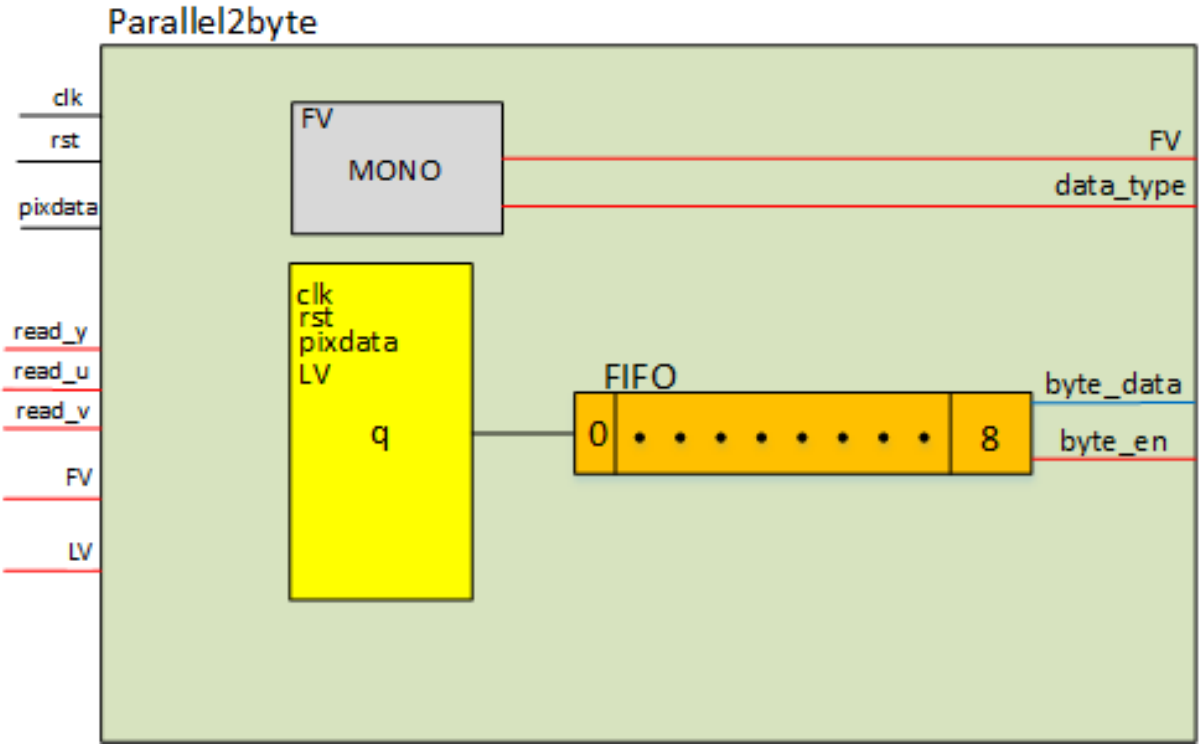

Figura 16 – *Circuito que agrupa os bytes em pacotes*

3.2.3.2 Circuito para adicionar cabeçalhos

A função deste circuito é anexar aos dados um cabeçalho contendo as informações necessárias para que o receptor possa decodificá-los no padrão correto : Número de lanes usadas, *word_count* e *data_type*, são as informações contidas no cabeçalho do long packet. Este bloco também formata um índice para o short packet, indicando quando a transmissão começará.

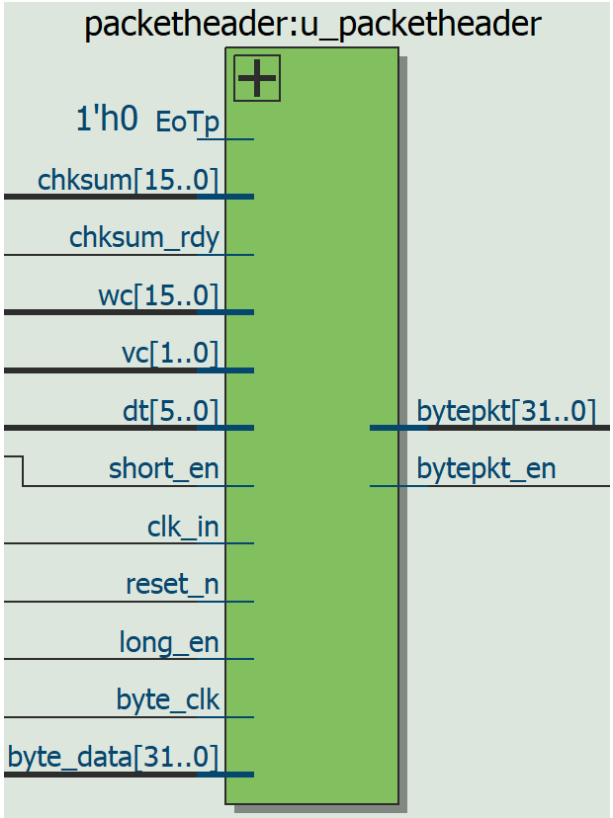

Figura 17 – *Circuito que anexa os cabeçalhos*

O circuito é ordenado por uma máquina com 8 estados, onde os quatro primeiros estados são referentes ao *short packet* e os últimos referentes ao *long packet* que compõe o low level protocol.

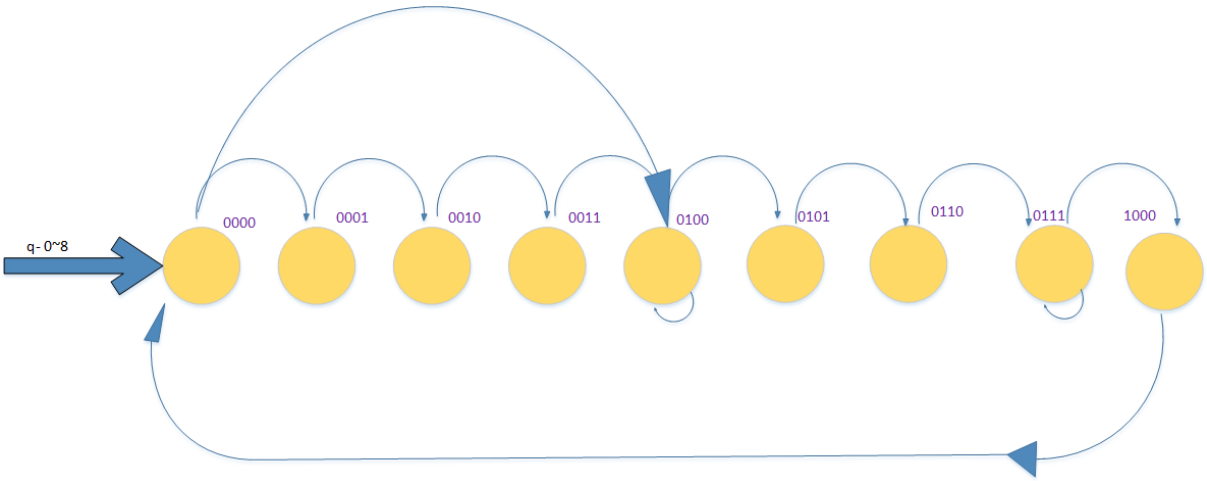

Figura 18 – *Demonstração da máquina de estados que comanda o módulo Packet Header*

Na máquina de estados, além de adicionar os cabeçalhos referente a cada tipo de pacote que será tansmitido, ela também contém pacotes prédeterminados para marcar o

ínicio e o fim de transmissão da *stream* de dados.

A saída do circuito packetheader será em pacotes de 32 bits, porém, o módulo composto por ele e pelo outro subcircuito terá quatro saídas, de 8bits cada. Destas, duas não terão dados e duas carregarão os dados da aplicação, isto devido ao fato do formato CSI-2 ter disponível até 4 lanes de dados e neste caso ter sido escolhido usar 2 lanes.

3.2.4 Circuito controlador de atrasos

Este circuito tem o encargo de controlar o delay entre o clock e as linhas de dados. O delay adicionado é necessário para que ocorra a transmissão simultânea no estado alto, quanto no estado baixo do pulso de clock. Neste módulo, será criado um sinal de enable para identificar quando a transmissão de dados é válida, acompanhando os sinais serializados no modo diferencial, com este bit, é possível ter uma visão mais ampla do formato do low level protocol, onde tem-se um *short packet* seguido de um *long packet*.

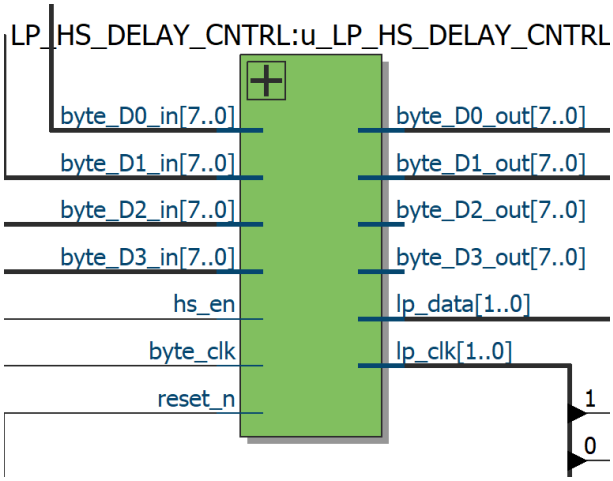

Figura 19 – *Módulo que irá controlar os atrasos necessários para sincronismo*

3.2.5 Circuito representante da camada física

É o último módulo do circuito. Este possui a função de serializar os bytes do circuito em bits e transformá-los em sinais diferenciais. É composto por 2 circuitos internos, o *LVDS_TX* e o *IOBUFF*.

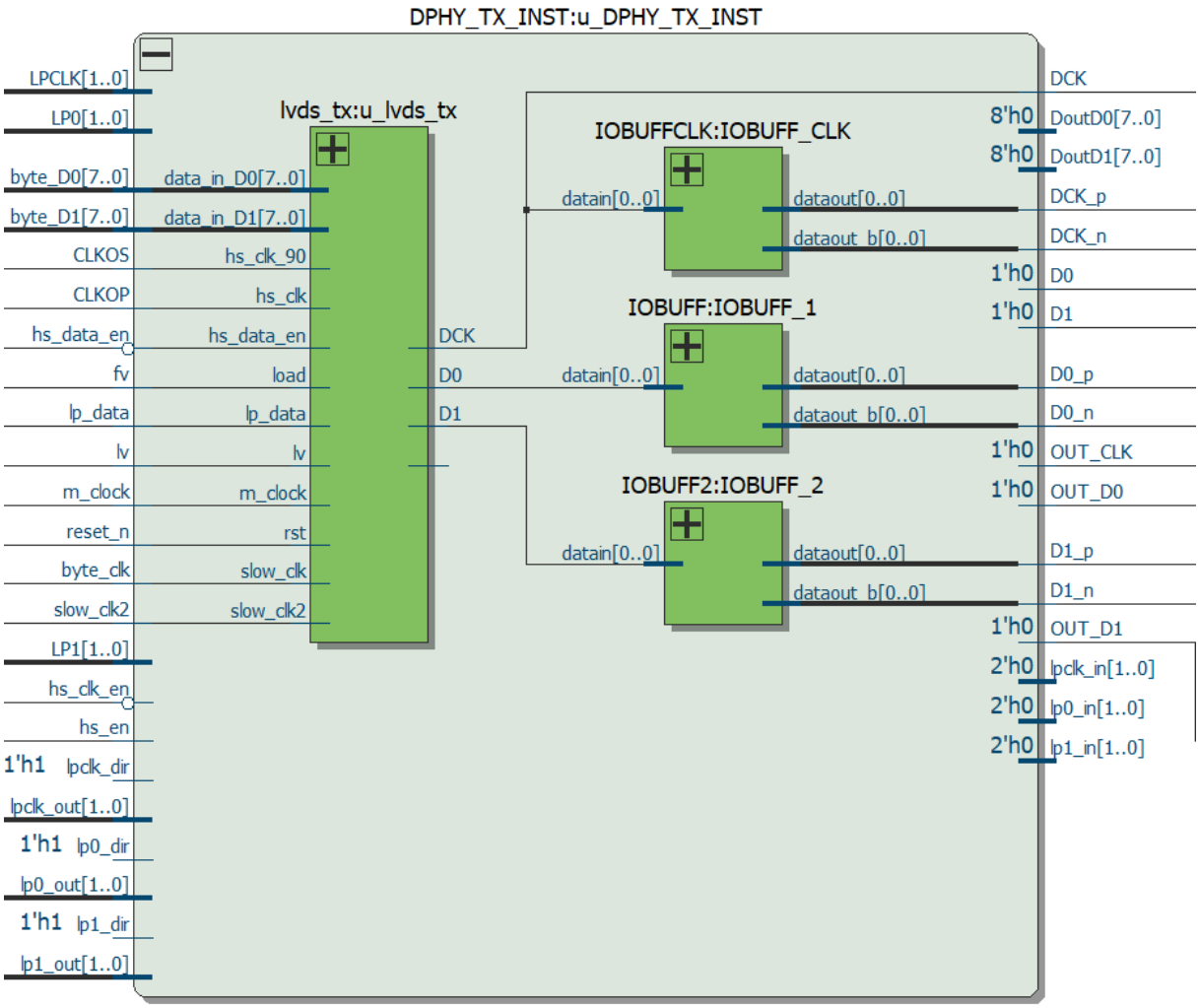

Figura 20 – *circuito representante da conexão com a camada física*

3.2.5.1 Circuito serializador dos dados

Até então, os dados estavam sendo transferidos de um módulo para outro no formato paralelo,este circuito é responsável por transformar uma entrada de dados paralelo em uma saída serial, o último passo do codificador CSI-2. Para que esta mudança seja possível, este circuito faz uso de um clock de saída oito vezes mais rápido que o de entrada, de forma que um dado de 8bits possa ser transmitido em 8 pulsos do clock de saída no mesmo tempo que o clock de entrada teve apenas uma comutação, assim não há perda de informação.

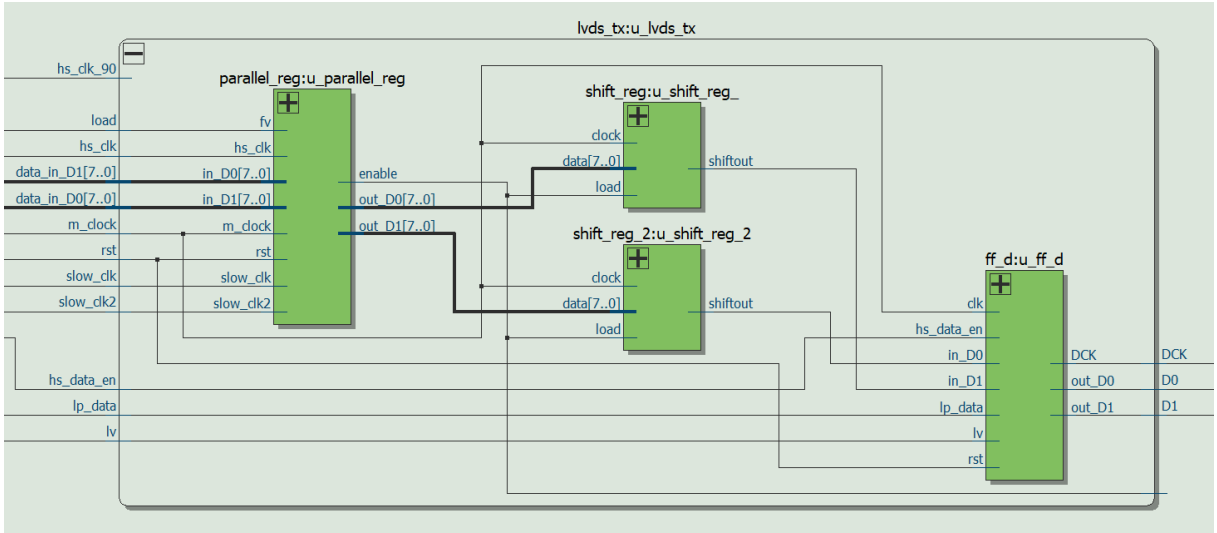

Figura 21 – *Módulo serializador de dados*

Este módulo é composto por um circuito registrador de dados, onde todos os 8 bits são registrados em registradores separados e estes mesmos 8 bits são a entrada de uma IP, chamada *SHIFT_REG*, que serializará o byte de entrada em bits. Feita esta ação, os bits são passados em um flip-flop D e encaminhados para fora do módulo.

3.2.5.2 Circuito para realizar conversão do formato serial para diferencial

Este circuito transformará os dados serial no padrão diferencial. É uma IP que já está habilitada no software quartus, foi usada para as duas saídas de dados do circuito e para o sinal de clock que as acompanha.

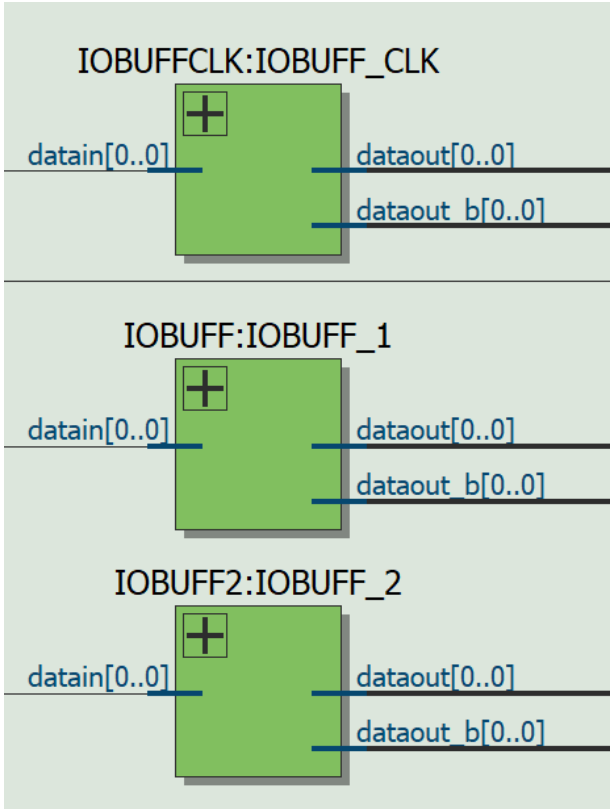

Figura 22 – *Circuitos gerados pelo IP MEGAWIZARD do software Quartus II*

4 Resultados

Os resultados demonstrados aqui, tem o objetivo de verificar o funcionamento do protocolo CSI-2 através da análise dos pacotes de bytes que estão sendo enviados em, 1 ou 2 linhas de dados.

Em relação ao número de linhas, a tabela 3 mostra a quantidade de elementos lógicos gastos para compilar o circuito projetado. O número de elementos aumenta de acordo com o decréscimo do número de linhas, pois para implementar a transmissão dos pacotes em menos canais de comunicação é usado um número maior de combinações lógicas dentro da FPGA.

Tabela 3 – Elementos Lógicos

Linha	Elementos Lógicos
1	1,562 / 114,480 (1 %)
2	1,589 / 114,480 (1 %)

4.1 Simulação com o Signal Tap

4.1.1 Interface com 1 linha de comunicação

A figura 23 foi retirada do *software Signal Tap*, nela estão presentes os dados no formato paralelo em hexadecimal, seguindo o padrão especificado pela *MIPI Alliance*, onde os campos referentes a: *Word Count, Data Type e Virtual Channel* devem ser transferidos do bit menos significativo para o mais significativo.

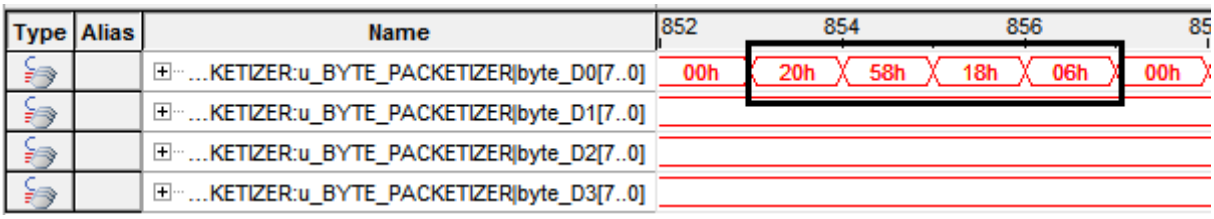

Figura 23 – *Imagem retirada do Signal Tap - Dados em paralelo presentes apenas em uma linha de comunicação*

Os dados em paralelo antes de serem transmitidos precisam ser convertidos para o formato serial, figura 24. Isto é feito ao projetar-se um circuito com um *clock* de saída com velocidade maior que o de entrada, caracterizando assim uma das vantagens do protocolo CSI-2, altas taxas de transferência de dados.

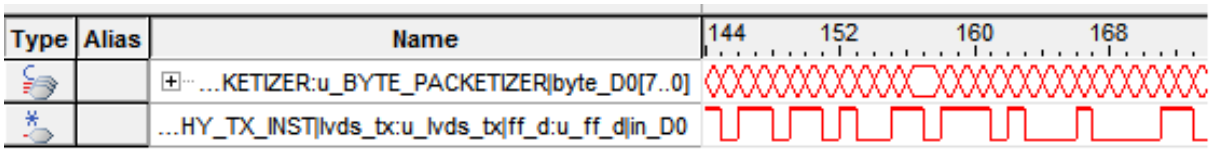

Figura 24 – *Imagem retirada do Signal Tap - Dados em paralelo presentes apenas em uma linha de comunicação*

4.2 Interface com 2 linhas de comunicação

Ao realizar as modificações necessárias no projeto do circuito, foi possível simular a transmissão de dados em 2 linhas de comunicação, figura 25. Agora os campos de cabeçalhos são divididos nas duas lanes, de forma que elas juntas carreguem a mesma informação que seria carregada pelo circuito com 1 ou 4 lanes.

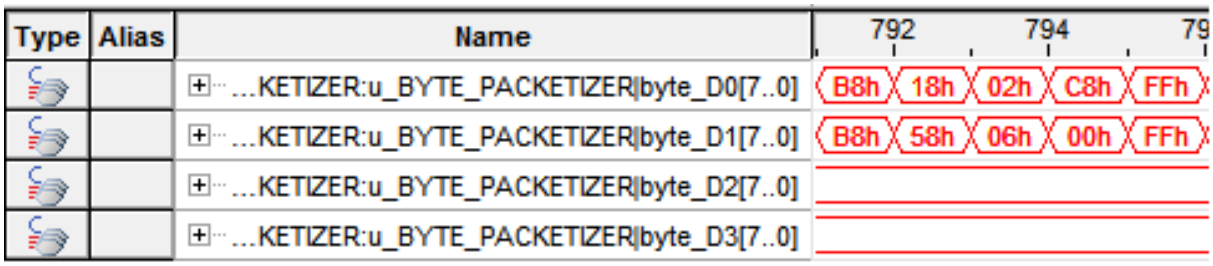

Figura 25 – *Imagem retirada do Signal Tap Logic AnalyzerDados entrando em paralelo com os cabeçalhos*

Os dados das duas linhas no formato paralelo com os cabeçalhos em destaque, figura 26.

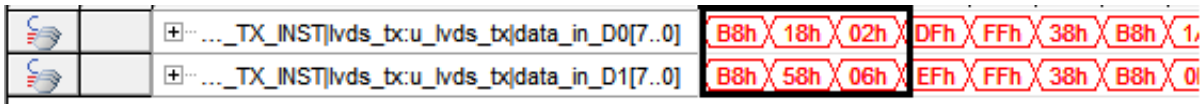

Figura 26 – *Imagem retirada do Signal Tap Logic Analyzer - dados do cabeçalho em paralelo e a saída no formato serial*

Após a adição do cabeçalho, os dados são serializados para serem enviados, figura 27 e mantêm o padrão de imagem estabelecido.

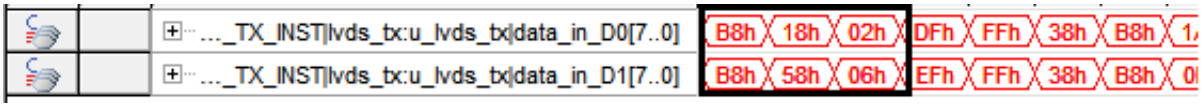

Figura 27 – *Imagem retirada do Signal Tap Logic Analyzer - Dados serializados*

5 Conclusão

O funcionamento do protocolo implementado na *FPGA* foi comprovado devido a análise dos pacotes ao longo dos módulos da interface para as duas opções de canais de comunicação. As simulações foram feitas visando uma saída na frequência de operação de 200Mhz, esta saída pode ser alterada de acordo com os requisitos do usuário, visto que o protocolo tem como característica uma transmissão em alta velocidade, podendo atingir valores superiores aos testados. Outros parâmetros também podem ser alterados, como o tipo de dado a ser transmitido ou a quantidade de bytes por pixel, sem que seja necessário realizar alterações no hardware de um projeto.

Por fim, a interface de transmissão serial padronizada pela *MIPI Alliance* prova sua eficiência em termos de velocidade e fornece a garantia que poderá ser usada para diversos sensores de imagem, desde que os canais de comunicação entre o sensor e o processador estejam dentro das especificações *CSI-2*.

5.1 Trabalhos Futuros

Como sugestão para novos trabalhos,tem-se o desenvolvimento de um circuito decodificador deste protocolo, seguindo as mesmas especificações de forma que seja possível trabalhar com imagens e vídeos de diversas resoluções, além do envio de pacotes individuais de dados.

REFERÊNCIAS

[1] *MIPI Alliance specification for Camera Serial Interface 2 (CSI - 2).* , 2009.

[2] Gordon, B. ; Chaddha, N. ; Meng, T H -Y.*Artigo: A low power multipliers YUV to RGB converter based on human vision perception.* Publicado em *Proceedings of 1994 IEEE Workshop on VLSI Signal Processing* em 26-28 de Outubro, 1994.

[3] Zhao, J. *Video/Image processing on FPGA.* Degree of of Master of Science in Electrical and Computer Engineering - Worcester Polytechnic Institute. (WPI). Worcester, 2015.

[4] Gamal, A E e Eltoukhy, H. *CMOS Image Sensors* IEEE Circuits and Devices Magazine (Volume: 21, Issue: 3), Maio e Junho, 2005.

[5] Neuendorffer, S e Vissers K. *Streaming Systems in FPGAs*,completar

[6] Noronha,D B ;Fountoura, K L;Júnior, J B V. *Uma vião geral sobre dispositivos lógicos reconfiguráveis(FPGAs) e suas aplicações.*,Universidade Federal de Uberlândia (UFU). Minas Gerais, 2005.

[7] Roth, F. *Using Low Cost FPGAs for realtime video processing Master's Thesis - Masaryk University - Faculty of Informatics* (MU). Tchéquia, 2011.

[8] Wang, P e McAllister, J. *Streaming Elements for FPGA Signal and Image Processing Accelerators* IEEE Transactions on Very Large Scale Integration (VLSI) Systems, volume 24, 2016.

[9] Petroni, J D S. *Interface para processamento de imagens implementada em FPGA*, Universidade de São Paulo (USP), São Carlos, 2015.

[10] Etheredge C E.*"GoSlow: Design and Implementation of a scalable camera array for high-speed imaging".Faculty of Electrical Engineering, Mathematics and Computer Science*, University of Twente. Julho, 2016.

[11] Mehta S; Patel A; Mehta J. *CCD or CMOS Image Sensor For Photography , International Conference on Communications and Signal Processing (ICCSP).* Abril, 2015.

REFERÊNCIAS